CLAUDE CERTIFIED ARCHITECT — PROFESSIONAL

Objective 37: Improve Developer Workflows Using AI-Assisted Tooling

Exam objective: Apply Claude Code commands, skills, and automation patterns to improve shared development workflows, including testing, code review, CI/CD integration, and measurable productivity gains across a team.

Q1 **A retail company's engineering team wants to standardize a repeatable "generate a changelog entry" workflow so any developer can trigger it the same way without re-explaining the format every time. What Claude Code feature is best suited for this?**

- **A** **A custom slash command that encapsulates the changelog-generation prompt and steps**
- **B** A one-off chat message typed fresh each time
- **C** A managed settings permission rule
- **D** An MCP server dedicated solely to changelogs

CORRECT ANSWER **A**

WHY Custom commands let teams package a reusable, consistently-invoked workflow (like changelog generation) so any developer gets the same behavior without retyping instructions. Permission rules govern access/safety not workflow content, and a dedicated MCP server is unnecessary overhead for a prompt-based task.

Q2 **A logistics company wants to give Claude Code the ability to generate database migration scripts following their specific schema conventions, with supporting reference templates and examples that are only loaded into context when that particular task is relevant. What should they build?**

A. A single giant CLAUDE.md file containing every possible convention for all tasks

B. A Skill that packages the migration-writing capability with its own instructions and reference files, progressively loaded only when needed

C. An MCP server that replaces Claude Code entirely

D. A permission rule granting broader file access

CORRECT ANSWER B

WHY Skills are designed exactly for this — reusable capabilities with bundled instructions/resources that load progressively only when relevant, keeping context efficient rather than bloating every session with unrelated conventions. A single giant CLAUDE.md wastes context on unrelated tasks, and permissions don't carry workflow logic.

Q3 **An engineering manager wants evidence that adopting Claude Code across the team is actually improving velocity, not just activity. Which approach best supports measuring developer productivity impact?**

A. Counting the number of Claude Code sessions started per day

B. Tracking outcome-oriented metrics such as PR cycle time, review turnaround, and defect rates before and after adoption, correlated with Claude Code usage

C. Asking developers if they "feel" more productive with no further data

D. Measuring the number of lines of code Claude Code generates

CORRECT ANSWER B

WHY Outcome metrics like cycle time, review turnaround, and defect rates tied to actual business value give a credible productivity signal, whereas raw session counts or lines-of-code are vanity metrics that don't reflect quality or throughput. Subjective feelings alone lack the rigor needed for a management decision.

Q4 **A fintech startup wants Claude Code to automatically run their unit test suite after every code change it makes during a session, and to halt further edits if tests fail, without depending on the model choosing to do so. How should this be implemented?**

A Add a note in CLAUDE.md asking Claude to run tests after changes

B **Configure a post-tool-use hook that runs the test suite after file edits and surfaces/blocks on failures**

C Rely on the developer to manually run tests after the session ends

D Use a Skill with no automation attached

CORRECT ANSWER B

WHY A post-edit hook deterministically triggers the test run every time, independent of model judgment, and can be used to block/alert on failure — this is the reliable automated-testing integration pattern. CLAUDE.md guidance is advisory only, and manual runs or an unattached skill don't provide automatic enforcement.

Q5 **A software team wants Claude Code to assist with code review by flagging potential issues in a pull request, but they want this review assistance to run automatically in their CI pipeline rather than requiring a developer to invoke it locally. What is the appropriate integration pattern?**

A Have each developer manually paste PR diffs into a local Claude Code session

B **Integrate Claude Code into the CI/CD pipeline as a controlled automation step that reviews the diff and posts findings, governed by appropriate permissions and scoped credentials**

C Disable all permission checks in CI to simplify the process

D Avoid CI integration since Claude Code only works interactively

CORRECT ANSWER B

WHY Claude Code can run non-interactively as part of a CI/CD pipeline, provided it's configured with proper scoped credentials and permissions, which automates review consistently for every PR rather than relying on manual, inconsistent local invocation. Disabling permission checks in CI is a security anti-pattern even though automation is otherwise appropriate.

Q6 **A platform team notices that different squads have each independently reinvented similar Claude Code commands for scaffolding new microservices, leading to inconsistent output. What is the best way to drive team-wide adoption of a single standard?**

A Let each squad continue independently since autonomy is more important than consistency

B **Centralize the scaffolding logic into a shared Skill or command distributed via a shared/managed configuration location, with documentation encouraging reuse**

C Ban the use of custom commands entirely

D Require a manual design review for every scaffold generated

CORRECT ANSWER **B**

WHY Centralizing the reusable capability as a shared Skill or command (distributed through shared or managed configuration) prevents duplicated, divergent effort and drives consistent adoption across squads. Banning commands removes a productivity tool, and manual review for every output doesn't scale and doesn't address the root duplication problem.

Q7 A healthcare SaaS company wants new engineers to onboard faster by using Claude Code to explain the existing codebase's architecture interactively. Which combination of features best supports this shared workflow?

- **A** **A CLAUDE.md file describing architecture/context plus a custom command that walks through key modules, so every new hire gets the same guided experience**
- **B** Giving new hires root access to production and letting them explore directly
- **C** A single all-access permission rule with no project context
- **D** Requiring new hires to read the codebase for a month before touching Claude Code

CORRECT ANSWER **A**

WHY Combining persistent project context (CLAUDE.md) with a reusable custom command creates a consistent, repeatable onboarding workflow applicable to every new hire. Production access, broad permissions, or a purely manual reading period don't leverage Claude Code's workflow capabilities at all.

Q8 A team lead wants to reduce the time spent on repetitive PR description writing. Developers often forget to mention breaking changes or migration steps. What AI-assisted workflow improvement addresses this most directly?

- **A** **A shared command/skill that generates PR descriptions from the diff, prompting explicitly for breaking-change and migration notes as part of its template**
- **B** Telling developers verbally to "try harder" to remember
- **C** Removing the PR description field entirely
- **D** Switching to a different version control system

CORRECT ANSWER **A**

WHY A structured, reusable command/skill that explicitly prompts for the commonly-missed sections (breaking changes, migration steps) systematically closes the gap rather than depending on individual memory. Removing the field or switching tools doesn't solve the underlying workflow problem.

Q9 **An e-commerce company's CI pipeline uses Claude Code non-interactively to auto-triage failing test logs and suggest fixes. The security team wants this automation to never be able to push directly to the main branch, only open draft PRs. How should this be configured?**

A Give the CI automation full repository admin credentials for flexibility

B **Scope the CI service credentials/permissions narrowly so the automation can only create draft PRs/branches, never push to protected branches, combined with permission rules restricting relevant tool actions**

C Trust the AI not to push to main without any technical restriction

D Run the automation with no credentials at all so it cannot do anything

CORRECT ANSWER B

WHY Controlled AI-assisted automation in CI/CD requires least-privilege scoped credentials and permission rules enforcing the intended boundary (draft PRs only, no direct push), which is a deterministic technical control rather than trust-based. Full admin credentials or zero credentials are both misconfigurations relative to the stated goal.

Q10 **A data engineering team wants to roll out a shared library of Claude Code skills (SQL optimization, data validation, pipeline scaffolding) across 15 teams, but wants usage data to know which skills are actually valuable before investing further. What should they prioritize?**

A Build every possible skill up front with no usage feedback loop

B **Launch a focused initial skill set, instrument/monitor usage and outcomes, then iterate based on observed adoption and productivity impact**

C Mandate skill usage via a policy with no measurement

D Avoid building skills until every team agrees unanimously on requirements

CORRECT ANSWER B

WHY An iterative rollout with usage monitoring lets the team validate value and refine the skill library based on real data, which is a pragmatic adoption strategy for scaling across many teams. Building everything up front, mandating without data, or waiting for full consensus all delay or waste investment relative to an evidence-based approach.

Q11 **A team wants a "release notes" command and a "security audit" skill to both be available in every repository in their organisation, maintained centrally so updates propagate automatically. Where should these be defined?**

A Copy-pasted manually into each repository's local folder by each team

B **Defined once in a shared/managed location (e.g., organisation-level or managed settings-distributed configuration) so updates propagate centrally rather than being duplicated per repo**

C Emailed as text snippets to each team lead

D Hardcoded into the Claude model weights

CORRECT ANSWER B

WHY Centralizing commands/skills in a shared, organisation-distributed configuration location ensures a single source of truth that propagates updates automatically, avoiding the drift and maintenance burden of per-repo copies. Email distribution or expecting changes to model weights are not valid configuration mechanisms.

Q12 **A software vendor integrates Claude Code into their CI to automatically generate test cases for newly added functions. Leadership asks how to validate that this is actually improving code quality rather than just increasing test count. What should be measured?**

A Only the raw number of new tests generated

B **Meaningful quality signals such as code coverage of critical paths, defect escape rate, and whether generated tests catch real regressions over time**

C The length of the generated test files

D How quickly Claude Code generates the tests with no regard to correctness

CORRECT ANSWER B

WHY Quality-oriented signals — coverage of critical logic, reduced defect escape rate, and real regression catches — demonstrate actual value, whereas raw test count or file length are superficial and can be gamed without improving quality. Generation speed alone ignores whether the output is useful.

Q13 **A team currently has each developer manually asking Claude Code to "review this diff for security issues" with slightly different wording each time, producing inconsistent depth of review. What's the most effective fix?**

A Leave it as-is since variation encourages creative reviews

B **Create a standardized custom command or skill with a consistent, well-tested security review prompt/checklist that all developers invoke the same way**

C Forbid security-related prompts entirely

D Require legal sign-off before any review prompt is used

CORRECT ANSWER B

WHY A standardized, reusable command/skill ensures every invocation follows the same vetted checklist, producing consistent review depth and quality across the team instead of ad hoc phrasing. Forbidding the practice or requiring legal sign-off for routine prompts is disproportionate and removes a useful productivity tool.

Q14 **An organisation wants developers to be able to trigger a "deploy to staging" workflow via Claude Code, but wants the actual deployment step to require a human to click approve in the CI system rather than happening autonomously. How should this be designed?**

A Let Claude Code trigger deployment directly with no gate, trusting the automation completely

B **Design the Claude-assisted workflow so it prepares/queues the deployment but the CI/CD pipeline includes a manual approval gate before the deploy step executes**

C Remove Claude Code from the workflow entirely since any AI involvement is too risky

D Have Claude Code ask the question once per quarter instead of per deployment

CORRECT ANSWER **B**

WHY Keeping a human-in-the-loop approval gate in the CI/CD pipeline while still letting Claude Code prepare/assist the workflow balances productivity gains with controlled automation — this is the standard pattern for safe AI-assisted deployment workflows. Fully autonomous deployment removes necessary oversight, while excluding Claude Code entirely discards legitimate productivity value.

Q15 **A consulting firm wants to track whether their investment in Claude Code skills and commands is paying off across client engagement teams. Which combination of indicators would best demonstrate ROI to leadership?**

A Anecdotes from a single enthusiastic developer

B **A combination of adoption rate across teams, time saved on repeatable tasks (e.g., scaffolding, review, documentation), and downstream quality/delivery metrics**

C The total dollar amount spent on API usage with no outcome context

D The number of Slack messages mentioning "Claude"

CORRECT ANSWER **B**

WHY Combining adoption breadth, measurable time savings on specific repeatable tasks, and downstream delivery/quality metrics provides a defensible, multi-dimensional ROI picture for leadership. Anecdotes, raw spend without outcomes, or informal mention counts are weak, non-rigorous indicators.

Q16 A platform team repeatedly needs a specialized "database migration reviewer" capability: a narrower toolset, a dedicated system prompt focused on migration safety, and its own isolated context so it doesn't clutter or get confused by the main conversation's history. Which Claude Code feature best fits this need?

A **A subagent defined in `.claude/agents/` with its own scoped tools, prompt, and context, invoked for this specific delegated task**

B Appending the migration-review instructions to the project's main CLAUDE.md so they always load

C A managed-settings permission rule granting broader database access

D A single long-lived chat session kept open indefinitely for all migration work

CORRECT ANSWER **A**

WHY Subagents are purpose-built for delegated, specialized work that needs its own tools, prompt, and isolated context — invoking one keeps the main session's context clean and focused while still providing the specialized capability on demand. Adding it to the main CLAUDE.md would load it into every session regardless of relevance, a permission rule only governs access rather than behavior, and a single persistent session doesn't provide isolation or reuse across tasks.

Q17 A team wants Claude Code to automatically review every pull request in CI and post structured comments as a bot, but is concerned that if the same session that authored the code also performs its own review, it will share the same blind spots and miss its own mistakes. How should they configure this safely while still automating the review?

A **Run Claude Code in headless mode in the CI pipeline (e.g., `claude -p` with JSON output and a scoped tool allowlist) as a separate, independent review step distinct from the authoring session, with humans still approving the final merge**

B Let the same session that wrote the code also approve and merge its own pull request to save time

C Avoid CI integration entirely and have developers manually copy diffs into a fresh chat each time

D Run an interactive Claude Code session on a shared server that the whole team takes turns using for reviews

CORRECT ANSWER **A**

WHY Headless, non-interactive execution with structured JSON output and a scoped tool allowlist is the supported pattern for automated CI review, and running it as an independent step — separate from whatever session authored the change — avoids the blind-spot problem of a session reviewing its own work, while a human approval gate preserves oversight. Letting the authoring session review itself defeats the purpose of independent review, and manual copy-paste or shared interactive sessions are not automated, consistent patterns. --